Nifty AI Options Trading Agent — Complete Master Plan v3

> Feed this entire document to Claude AI in VS Code to resume from exactly where we left off. Claude should WRITE code. YOU run it. Never ask Claude to execute — saves tokens.

QUICK START FOR VS CODE CLAUDE

Paste this file and say: **"I have read the master plan v3. Let's start Phase 1 Step 1: create the nifty_agent/ folder structure."**

Critical facts to never forget:

- Nifty lot = **65** (NOT 75)
- BankNifty lot = **30** (NOT 35)
- Sensex lot = **20** (BSE)
- Nifty expiry = **TUESDAY** (NOT Thursday, changed Sept 2025)
- Sensex expiry = **THURSDAY** (BSE)
- All orders = **MARKET + SL-M** (no limit order logic anywhere)
- Fyers = **data only**, Groww = **execution only**
- KILL_SWITCH_ACTIVE = **never cache**, always read DB directly
- Model decides SL, trail, phase targets — **no hardcoded values**
- India VIX = **primary driver** for SL and trail width decisions

---

SECTION 1 — PROJECT OVERVIEW

Goal: Fully autonomous AI options trading agent for NSE + BSE India.

Core philosophy:

- Train on what we trade (option premiums), not just what drives it (index)
- Multi-timeframe patterns humans cannot track
- Zero emotion, consistent execution
- Model improves weekly from its own trade data
- Model owns SL, trail, and phase decisions — humans own risk rails only

Brokers:

- Fyers API (FREE) → historical data + live WebSocket feed
- Groww API (₹499/mo) → order execution only

Order Execution:

- Entry: MARKET order → fills instantly at best available price
- SL Exit: SL-M order placed immediately after entry confirmation
- Exchange handles SL execution — survives EC2 crashes natively
- No limit order repricing, no retry loops, no GTT backup needed

Infrastructure:

- Local machine → Model Improver (data + ML, no EC2 needed)
- AWS EC2 + Elastic IP → Live Trade service
- Railway or local PostgreSQL → database

SECTION 2 — EXISTING REPOSITORIES

Repo 1 — nifty_ml_model

URL: https://github.com/dellindone/nifty_ml_model

What exists:

- Direction prediction model: +1 (bullish), 0 (neutral), -1 (bearish)
- Features: RSI, MACD, ATR, Bollinger, India VIX, regime detection
- 6 strategies: model_only, trend_following, mean_reversion, hybrid_alpha, precision_alpha, regime_adaptive

- Walk-forward validation already implemented
- Live signal scaffold (Groww WebSocket)
- Saves: nifty_direction_model.joblib, metrics.json, backtest.json, research_report.json

Problems to fix:

- Uses yfinance → only 60 days intraday (useless for 3yr training) → replace with Fyers
- Single timeframe only → need multi-TF
- No option premium data in training
- No VIX features for SL/trail learning
- Signals go directly to Groww → should go through monitoring service
- Groww live feed connected directly → decouple

Files to reuse (copy to core/):

- `src/nifty_prediction_model/features.py` → core/features/engineering.py (70%)
- `src/nifty_prediction_model/regime.py` → core/features/regime.py (100%)
- `src/nifty_prediction_model/model.py` → core/model/train.py (80%)
- `src/nifty_prediction_model/strategy.py` → core/strategy/ (70%)
- `scripts/research_model.py` → scripts/research_model.py (90%)

Repo 2 — monitering_service

URL: https://github.com/dellindone/monitering_service

What exists (production-grade):

- FastAPI + async PostgreSQL + Groww broker adapter
- Full SOLID principles, 8 design patterns
- Trailing SL engine (configurable % steps)
- Kill switch (daily loss + daily target limits)
- External trade detection (scans Groww positions every 60s)

- TradingView webhook at POST /webhook/signal
- State machine: PENDING → OPEN → SL_HIT → CLOSED → FAILED
- Telegram bot already configured
- Docker + Railway ready

Key files:

- `config.py` — pydantic BaseSettings, lot_size_multiplier per instrument, lru_cache
- `core/database.py` — async SQLAlchemy, asyncpg, SSL, pool_size=5
- `models/trade.py` — Trade + DailyStat ORM models
- `brokers/groww/adapter.py` — GrowwAdapter (place_order, get_positions, WebSocket)
- `brokers/groww/auth.py` — GrowwAuth singleton, TOTP
- `engine/trade_manager.py` — singleton, all open trades
- `engine/trade_monitor.py` — observer per trade, triggers SL exit
- `engine/stoploss_strategy.py` — trailing SL bands
- `engine/trade_command.py` — BuyCommand, SellCommand
- `risk/daily_risk_manager.py` — kill switch + daily P&L tracking

Problems to fix:

- Only accepts TradingView webhooks → add /signals/ml endpoint
- Uses MARKET orders already — keep as-is ✅
- No confidence filter → add MIN_CONFIDENCE gate
- SL execution → replace software trailing with SL-M amend pattern
- No multi-leg for option selling → add later

Files to reuse (copy to live_trade/):

- `brokers/` → live_trade/brokers/ (100%)
- `engine/` → live_trade/engine/ (update SL to SL-M amend pattern)
- `risk/` → live_trade/risk/ (100%)

- `repository/` → live_trade/repository/ (100%)
- `core/database.py` → live_trade/core/database.py (100%)
- `models/trade.py` → live_trade/models/trade.py (extend with 12 new fields)

Repo 3 — chartflix-backend

URL: https://github.com/dellindone/chartflix-backend

What exists:

- Fyers API: 5-step TOTP login, singleton, 20hr expiry, auto re-login
- TOTP timing guard (sleeps at 27-30 second boundary to prevent expired TOTP)
- Option chain: CSV fetch, Redis cache (6h TTL, gzip+pickle), expiry selection
- NSEBSEStrategy: ITM2 → ITM1 → ATM with spread filter (Mode B logic)
- StrategyFactory: NSE/BSE vs MCX routing
- FyersClient.invalidate() → auto re-login on auth error

Key raw URLs (fetch directly):

```
https://raw.githubusercontent.com/dellindone/chartflix-backend/main/app/services/
https://raw.githubusercontent.com/dellindone/chartflix-backend/main/app/services/
https://raw.githubusercontent.com/dellindone/chartflix-backend/main/app/services/
```

Files to reuse (copy to core/):

- `fyers_client.py` → core/data/fyers_client.py (85%, add 4 methods)
- `option_chain.py` → core/data/option_chain.py (92%, fix 4 things)
- `strategies.py` → core/strategy/strike_selector.py (90%, add mode awareness)

Changes needed for fyers_client.py:

- Add `get_history(symbol, date_from, date_to, resolution="15")`
- Add `connect_websocket(symbols, on_message)` using FyersDataSocket
- Add `schedule_morning_refresh()` async cron (8:45 AM daily)
- Add token file persistence (save/load .fyers_token JSON)

- Change settings import from `app.core.config` to `config.settings`

Changes needed for option_chain.py:

- Swap Redis → pluggable file cache (no Redis locally for Model Improver)
- Add `mode` param to `_select_expiry()` : Mode A = delta>0, Mode B = delta>5
- Fix key inconsistency: `lp` → `ltp` throughout
- Make `MAX_INVESTMENT` configurable via DB config (currently hardcoded ₹40,000)

Changes needed for strategies.py:

- Add `mode` param (MODE_A vs MODE_B)
- Mode A: ATM or ITM1, premium ₹80-200
- Mode B: ITM2, premium must be > ₹300 (hard rule)
- ITM depth configurable (1 for Mode A, 2 for Mode B)

SECTION 3 — 2026 MARKET FACTS

Lot Sizes (January 2026 onwards)

```
NIFTY:       65   (was 75, changed Jan 2026)
BANKNIFTY:   30   (was 35, changed Jan 2026)
SENSEX:      20   (BSE)
FINNIFTY:    65   (verify from Fyers CSV)
MIDCPNIFTY:  75   (verify from Fyers CSV)
```

Expiry Days (September 2025 onwards)

```
NSE → TUESDAY (Nifty, BankNifty, FinNifty — changed from Thursday!)
BSE → THURSDAY (Sensex, BankEx)
Weekly: Nifty (every Tuesday) + Sensex (every Thursday)
No weekly: BankNifty (monthly only, last Tuesday)
```

Trading Schedule

```
Monday:    Nifty focus (2 days to Tuesday expiry)
Tuesday:   Nifty EXPIRY (Mode A test / Mode B fallback)
```

```
Wednesday: Sensex focus (1 day to Thursday expiry)
Thursday:  Sensex EXPIRY (Mode A test / Mode B fallback)
Friday:    Both indices, swing setups for next week
```

Charges Per Trade (April 2026)

```
STT (options sell): 0.15% of premium × lot_size
Brokerage Groww:    ₹20 per order (×2 = ₹40 round trip)
Exchange charges:   ~₹10 round trip
GST 18%:            on brokerage + exchange
Stamp duty:         0.003% buy side only
Slippage (market):  ₹1.0/unit ATM Nifty (typical)
                    ₹2.0/unit ITM Nifty
                    ₹1.5/unit ATM Sensex
                    ₹3.0/unit ITM Sensex

Nifty 1 lot, ₹150 premium:
  Premium deployed:   ₹150 × 65 = ₹9,750
  Charges (fixed):    ~₹74
  Slippage estimate:  ₹65 (₹1 × 65)
  Total round trip:   ~₹139 worst case
  Still within ₹1,000 minimum target ✅
```

SECTION 4 — FOLDER STRUCTURE

```
nifty_agent/
│
├── core/                              ← Shared by both repos
│   ├── data/
│   │   ├── fyers_client.py            ← FROM chartflix + 4 additions
│   │   ├── option_chain.py            ← FROM chartflix + 4 fixes
│   │   ├── option_premium_history.py  ← NEW: 3yr ATM option 15-min data
│   │   ├── synthetic_premium.py       ← NEW: BSM synthetic (Phase 1 fallback)
│   │   ├── multi_tf_builder.py        ← NEW: all timeframes aligned
│   │   ├── fii_dii.py                 ← NEW: daily FII/DII from NSE
│   │   └── gift_nifty.py              ← NEW: pre-market GIFT Nifty
│   │
│   ├── features/
│   │   ├── engineering.py             ← FROM nifty_ml_model (70%)
│   │   ├── regime.py                  ← FROM nifty_ml_model (100%)
│   │   ├── candlestick.py             ← NEW: TA-Lib 10 key patterns
│   │   ├── pattern_context.py         ← NEW: pattern + context scorer
```

```
│   │   ├── option_features.py        ← NEW: 23 features from CE/PE
│   │   └── vix_features.py           ← NEW: VIX regime + composite score
│   │
│   ├── model/
│   │   ├── train.py                  ← FROM nifty_ml_model (80%)
│   │   ├── research.py               ← FROM nifty_ml_model (90%)
│   │   ├── predict.py                ← NEW: live inference
│   │   ├── promoter.py               ← NEW: graduation evaluator
│   │   ├── calibrator.py             ← NEW: Platt scaling calibration
│   │   └── artifacts/                ← .joblib + metrics.json
│   │
│   ├── strategy/
│   │   ├── strike_selector.py        ← FROM chartflix strategies.py (90%)
│   │   ├── mode_selector.py          ← NEW: Mode A vs Mode B
│   │   ├── switcher.py               ← NEW: SWING vs SCALP auto-switch
│   │   ├── phase_manager.py          ← NEW: Hunt→Secured→Trailing phases
│   │   └── scalping_agent.py         ← NEW: unlocks at ₹3L capital
│   │
│   ├── risk/
│   │   ├── position_sizer.py         ← NEW: Kelly Criterion
│   │   ├── capital_tracker.py        ← NEW: capital + drawdown tracking
│   │   └── daily_risk.py             ← FROM monitering_service (100%)
│   │
│   ├── greeks/
│   │   └── calculator.py             ← NEW: mibian BSM
│   │
│   ├── notifications/
│   │   └── telegram.py               ← FROM monitering_service (adapted)
│   │
│   └── utils/
│       ├── symbol_converter.py       ← NEW: strip NSE:/BSE: prefix
│       ├── market_calendar.py        ← NEW: NSE holidays + expiry dates
│       ├── charge_calculator.py      ← NEW: exact charges + slippage
│       ├── liquidity_checker.py      ← NEW: bid-ask spread + OI gate
│       ├── slippage_estimator.py     ← NEW: per-instrument slippage budget
│       └── logger.py                 ← FROM monitering_service (100%)
│
├── model_improver/                   ← Git repo 1 (local only, no orders)
│   ├── engine.py                     ← main loop
│   ├── signal_handler.py             ← signal → validate → log
│   ├── journal.py                    ← parquet writer (40+ fields)
│   ├── shadow_mode.py                ← sends to EC2 without executing
│   ├── reporter.py                   ← weekly improvement report
│   └── data/
│       ├── trades.parquet            ← all paper trades
│       └── capital.parquet           ← capital history
│
```

```
├── live_trade/                        ← Git repo 2 (EC2)
│   ├── main.py                        ← FastAPI app
│   ├── api/
│   │   ├── ml_signal_router.py        ← NEW: POST /signals/ml
│   │   ├── config_router.py           ← NEW: GET/PATCH /config/{key}
│   │   ├── trades_router.py           ← FROM monitering_service
│   │   └── killswitch_router.py       ← FROM monitering_service
│   ├── services/
│   │   ├── ml_signal_service.py       ← NEW: confidence filter → BuyCommand
│   │   ├── manual_trade_service.py    ← NEW: POST /trades/manual
│   │   ├── webhook_service.py         ← FROM monitering_service
│   │   └── monitor_service.py         ← FROM monitering_service
│   ├── brokers/                       ← FROM monitering_service (100%)
│   │   └── groww/
│   │       ├── adapter.py             ← MARKET order + SL-M amend pattern
│   │       ├── auth.py
│   │       └── feed.py
│   ├── engine/                        ← FROM monitering_service
│   │   ├── trade_manager.py
│   │   ├── trade_monitor.py           ← updated: SL-M amend per candle
│   │   ├── stoploss_strategy.py       ← updated: model-driven SL/trail
│   │   └── trade_command.py           ← MARKET order only
│   ├── models/
│   │   └── trade.py                   ← FROM monitering_service + 12 new fields
│   ├── repository/                    ← FROM monitering_service (100%)
│   └── config/
│       ├── settings.py                ← secrets only (.env)
│       └── agent_config.py            ← DB-backed + AgentConfigService
│
├── scripts/
│   ├── download_data.py               ← one-time data download
│   ├── build_dataset.py               ← merge + features → parquet
│   ├── train_model.py                 ← train one pass
│   ├── research_model.py              ← multi-candidate sweep
│   └── weekly_retrain.py              ← cron: Sunday 6 PM
│
├── config/
│   ├── instruments.py                 ← lot sizes, expiry days
│   ├── settings.yaml                  ← model + strategy config
│   └── .env                           ← API keys (NEVER commit)
│
├── data/                              ← all parquet files
├── notebooks/
├── logs/
├── requirements.txt
└── .gitignore
```

SECTION 5 — INSTRUMENTS CONFIG

```
# config/instruments.py

LOT_SIZES = {
    "NIFTY":      65,
    "BANKNIFTY":  30,
    "SENSEX":     20,
    "FINNIFTY":   65,
    "MIDCPNIFTY": 75,
}

EXPIRY_WEEKDAY = {
    "NIFTY":      1,   # Tuesday
    "BANKNIFTY":  1,   # Tuesday (monthly only, no weekly)
    "FINNIFTY":   1,   # Tuesday (monthly only)
    "SENSEX":     3,   # Thursday
    "BANKEX":     3,   # Thursday
}

HAS_WEEKLY_EXPIRY = {
    "NIFTY":     True,
    "BANKNIFTY": False,
    "SENSEX":    True,
    "FINNIFTY":  False,
}

EXCHANGE = {
    "NIFTY":     "NSE",
    "BANKNIFTY": "NSE",
    "FINNIFTY":  "NSE",
    "SENSEX":    "BSE",
    "BANKEX":    "BSE",
}

STRIKE_GAP = {
    "NIFTY":     50,
    "BANKNIFTY": 100,
    "SENSEX":    100,
    "FINNIFTY":  50,
}

FYERS_SYMBOL = {
    "NIFTY":     "NSE:NIFTY50-INDEX",
    "BANKNIFTY": "NSE:NIFTYBANK-INDEX",
```

```
    "SENSEX":    "BSE:SENSEX-INDEX",
    "FINNIFTY":  "NSE:FINNIFTY-INDEX",
}
```

SECTION 6 — API KEYS

```
FYERS (data only, FREE):
  Role:   historical candles + live WebSocket + option chain CSV
  Auth:   5-step TOTP (already in chartflix fyers_client.py)
  Token:  expires daily at midnight → morning refresh at 8:45 AM
  Config: FYERS_CLIENT_ID, FYERS_SECRET_KEY, FYERS_REDIRECT_URI,
          FYERS_ID, FYERS_PIN, FYERS_TOTP_SECRET

GROWW (execution only, ₹499/mo):
  Role:   place_order (MARKET), place_slm_order, get_positions, WebSocket
  Auth:   API Key + TOTP
  Config: GROWW_API_KEY, GROWW_TOTP_SECRET

TELEGRAM (alerts):
  Config: TELEGRAM_BOT_TOKEN, TELEGRAM_CHAT_ID
  Sends:  entry, exit, drawdown, kill switch, daily summary, model promoted
```

SECTION 7 — DATA PIPELINE

Timeframes and History Available

```
Fyers resolution "1"  (1-min):   ~100 days hard limit
Fyers resolution "5"  (5-min):   ~1 year
Fyers resolution "15" (15-min):  ~3 years
Fyers resolution "60" (60-min):  ~7 years
Fyers resolution "1D" (daily):   ~20 years (also NSEpy free)
```

Option Premium Data Strategy

```
PHASE 1 (start training immediately):
  Use synthetic_premium.py
  Calculate theoretical option price using BSM:
    inputs: spot_price, atm_strike, dte, vix (as proxy for IV)
    output: synthetic CE + PE premium per 15-min bar
```

```
  Accuracy: 85-90%
  Zero extra API calls needed

PHASE 2 (improve after first model works):
  Use option_premium_history.py
  For each trading day (3 years = ~750 days):
    Find ATM strike from index close at 9:30 AM
    Construct option symbol (e.g. NSE:NIFTY25APR2222500CE)
    Download CE + PE 15-min bars from Fyers
  Total: 750 × 2 = 1,500 API calls (~15 minutes)
  Store: data/option_premiums.parquet (~3 MB)
```

Multi-Timeframe Feature Alignment

```
Base: 15-min bars (decision timeframe)

At EVERY 15-min bar, attach features from:
  1-min  last 15 bars → micro pattern (candle DNA)
  5-min  last 12 bars → short momentum
  15-min last 20 bars → base signal (regime, model output)
  60-min last 5 bars  → medium trend context
  1-day  last 20 bars → macro regime (bull/bear market)

No look-ahead bias rule:
  Higher TF feature at 10:15 = uses bar that CLOSED at 10:00
  NEVER uses the current bar that hasn't closed yet
```

SECTION 8 — FEATURE ENGINEERING (~175 total → select top 80)

Feature Categories

```
Index candle (25):
  OHLCV, body_ratio, upper/lower_wick_ratio
  volume_ratio_20, gap_from_prev_close
  position_in_day_range, consecutive_direction
  atr_14, momentum_3/5, ema_8/21, trend_direction
  vwap_position, prev_day_high/low_dist

Multi-timeframe (80):
  Same 20 features × 4 timeframes (1m, 5m, 60m, 1d)
```

```
  Prefixed: body_ratio_1m, momentum_5m, trend_60m, etc.

Candlestick patterns (13) [TA-Lib]:
  candle_hammer, candle_shooting_star
  candle_bull_engulfing, candle_bear_engulfing
  candle_morning_star, candle_evening_star
  candle_marubozu, candle_doji, candle_spinning_top
  candle_three_white, candle_three_black
  candle_bull_count, candle_bear_count, candle_net_signal

Option premium features (23):
  ce_premium, pe_premium, total_premium (straddle cost)
  ce_pct_change, pe_pct_change (1-bar momentum)
  ce_pct_change_3, pe_pct_change_3 (3-bar momentum)
  pcr_premium (pe/ce ratio → sentiment)
  ce_iv, pe_iv (implied volatility via mibian)
  iv_skew (pe_iv - ce_iv → downside fear)
  ce_delta, ce_theta, ce_gamma (Greeks via mibian)
  dte, dte_squared (theta decay is quadratic)
  theta_per_lot (daily ₹ decay per lot)
  ce_range_pct (option volatility)
  ce_vol_ratio, pe_vol_ratio (volume vs 20-bar avg)
  call_put_vol_ratio
  ce_leverage_ratio (option move / index move)
  premium_compression (IV crush flag = 1)

VIX features (8) [PRIMARY FOR SL/TRAIL LEARNING]:
  india_vix                  # raw VIX value
  vix_regime                 # 0=low(<12), 1=normal(12-18),
                             # 2=high(18-25), 3=extreme(>25)
  vix_percentile_52w         # VIX rank vs 52-week range (0-100)
  vix_change_1d              # VIX change vs yesterday
  vix_change_5d              # VIX trend over week
  vix_momentum               # VIX accelerating or decelerating
  vix_vs_option_iv           # VIX vs actual option IV (divergence)
  vix_mean_reversion         # distance from 20-day VIX mean
  volatility_composite       # weighted score: 0=calm, 1=extreme
                             # = 0.40×vix + 0.35×option_atr + 0.25×iv

Time features (10):
  hour, minute, day_of_week
  bars_since_open, bars_to_close
  bars_to_expiry, is_expiry_day
  is_monday, is_tuesday_nse, is_thursday_bse

Market context (8):
  india_vix (also in VIX features above)
```

```
  vix_change_pct
  fii_net_prev, dii_net_prev
  gift_nifty_premium (pre-market)
  oi_ce_atm, oi_pe_atm, pcr_oi

After SHAP feature selection: keep top 80-100
```

SECTION 9 — MODEL ARCHITECTURE

Three Models in Ensemble

```
MODEL 1 — XGBoost (primary)
  Input:  top 80 SHAP-selected features (includes VIX features)
  Output: direction (-1/0/+1) + class probabilities
          sl_bin (TIGHT/MEDIUM/WIDE/VERY_WIDE)
          trail_bin (TIGHT/MEDIUM/WIDE)
          phase1_target_bin (SMALL/MEDIUM/LARGE)
          trail_tf (1/5/15 minutes)
  Training: walk-forward validation

MODEL 2 — LSTM (sequence)
  Input:  last 20 × 15-min bars raw OHLCV + VIX
  Output: direction + probability + sl_bin + trail_bin
  Training: same walk-forward splits as XGBoost

MODEL 3 — Scalp Model (separate, 1-min events)
  Input:  1-min + 5-min features + VIX features
  Output: momentum event type + confidence + sl_bin
  Activates: only when capital > ₹3,00,000
  Events detected:
    first_candle_breakout (9:15-9:32 AM, body > 0.4%, vol > 2.5×)
    key_level_break (any time, level tested before, vol > 3×)
    vix_spike (VIX moves 3%+)
    gamma_squeeze (Thursday 1-3 PM, OI building fast)
    oi_buildup (unusual OI accumulation)

ORCHESTRATOR:
  Swing signal  = 0.6 × XGBoost + 0.4 × LSTM
  Scalp signal  = 1.0 × Scalp Model (separate decision)
  Priority: Expiry > Volatility > Trend > Reversion > Scalp
  Max simultaneous positions: 1 (start simple)
```

What Model Outputs (Per Signal)

```
# Model decides ALL of these — no hardcoding:
{
  "direction":       +1 / 0 / -1,
  "confidence":      0.0 to 1.0,
  "sl_bin":          "TIGHT" / "MEDIUM" / "WIDE" / "VERY_WIDE",
  "trail_bin":       "TIGHT" / "MEDIUM" / "WIDE",
  "phase1_target":   ₹X (learned from data),
  "trail_tf":        1 / 5 / 15,   # minutes
}

# Execution converts bins → actual ₹ using current ATR + VIX:
SL_BIN_TO_POINTS = {
    "TIGHT":     lambda atr, vix_score: atr × 0.8 × (1 + vix_score),
    "MEDIUM":    lambda atr, vix_score: atr × 1.2 × (1 + vix_score),
    "WIDE":      lambda atr, vix_score: atr × 1.8 × (1 + vix_score),
    "VERY_WIDE": lambda atr, vix_score: atr × 2.5 × (1 + vix_score),
}
```

SL/Trail Label Design (Training)

```
# Labels are distribution-based, not single-trade-based
# For each feature cluster (similar market states):
#   Collect all historical trades in that cluster
#   Find median SL that maximized profit in that cluster
#   Bin into category: TIGHT / MEDIUM / WIDE / VERY_WIDE
#   Use that as the supervised label

# VIX regime balance required in training:
assert regime_counts["LOW"]     >= 150
assert regime_counts["NORMAL"]  >= 300
assert regime_counts["HIGH"]    >= 100
assert regime_counts["EXTREME"] >= 30

# Key metric model minimizes:
# gave_back_pct = (max_profit_seen - exit_profit) / max_profit_seen
# Model learns: right SL + right trail = low gave_back_pct
```

Labels (Direction)

```
LABEL_SETTINGS = {
    "horizon_bars":     4,      # look 4 × 15-min ahead
    "move_threshold":   0.12,   # 12% option premium move
```

```
    "use_atr_scaled":   True,   # adapt to volatility
    "atr_multiplier":   1.5,
    "atr_window":       14,
}
# +1 if option gains >= threshold within horizon
# -1 if option falls <= SL_percent within horizon
# 0  if neither (no clear move)
```

Expected Win Rate Improvements

```
V1 — nifty_ml_model as-is:             55-58%
V2 — add multi-timeframe:              61-65%  (+6%)
V3 — add candlestick patterns:         63-67%  (+2%)
V4 — add pattern context scoring:      66-70%  (+3%)
V5 — add option premium features:      68-73%  (+3%)
V6 — add VIX-driven SL/trail learning: 70-75%  (+2% win rate
                                                 -15% gave_back_pct)
Total improvement over V1:             +15-17%
```

SECTION 10 — COMPLETE TRADING STRATEGY

Two Strategies, Auto-Switched

```
SWING STRATEGY (always active):
  Timeframe: 15-min bars
  Mode A: current expiry, ATM/slight OTM, ₹80-200 premium
  Mode B: 2 ITM next expiry, premium MUST > ₹300
  Max trades/day: 4
  Best window: 9:30-11:30 AM

SCALPING STRATEGY (unlocks at ₹3L capital):
  Timeframe: 1-min momentum events
  Max trades/day: 5
  DB gate: SCALPING_CAPITAL_GATE = 300000
  Auto-enables: when capital_tracker.current_capital >= gate
```

Mode A vs Mode B (Expiry Day)

```
MODE A — AI Volatility (test first):
  Current expiry, ATM or slight OTM
  ATR-based SL (model decides width via VIX composite)
```

```
  Full phase 1/2/3 system
  Test 8 weeks in Model Improver

MODE B — 2 ITM Logic (fallback):
  Skip current expiry entirely
  2 strikes ITM on NEXT expiry
  Premium MUST be > ₹300 (hard rule — if not found, NO TRADE)
  Trade 9:30 AM - 12:00 PM only on expiry day
  Max 2 trades on expiry day
  Phase 1 target: smaller % (higher delta = faster move)
  VIX regime check: HIGH/EXTREME VIX → always Mode B

AUTO-SWITCH RULES:
  After 8 weeks Model Improver data:
    Mode A expiry win rate < 52% → Mode B permanently on expiry
    VIX > 18 on expiry day → always Mode B
    Both tested in parallel during Model Improver phase
```

Phase System (Model-Driven, Not Hardcoded)

```
PHASE 1 — HUNT:
  Entry: MARKET order → fills instantly
  SL: entry_price - sl_points (model decides sl_points)
  SL-M placed on exchange immediately after fill
  Target: phase1_target (model decides ₹ amount)
  Trail timeframe: trail_tf minutes (model decides 1/5/15)

PHASE 2 — SECURED:
  Triggered: Phase 1 target hit
  Software: amend SL-M order to entry_price (breakeven)
  Exchange: SL-M now at breakeven
  Minimum profit locked in even if EC2 dies

PHASE 3 — TRAILING:
  Every trail_tf candle close:
    new_sl = candle_low - trail_points (model decides trail_points)
    if new_sl > current_sl:
        amend_slm_order(new_price=new_sl)  ← 1 API call
    else:
        do nothing  ← SL never moves backward

  After 1:30 PM: trail_tf always 1 (tightest, model override)
  Hard exit: 3:15 PM MARKET order always

VIX HARD RULES (override model, safety rails only):
  VIX > 30:  NO NEW TRADES
```

```
  VIX > 25:  force sl_points >= 35 (minimum)
  VIX < 12:  cap sl_points <= 20 (maximum)
  VIX intraday spike > 5%: amend current SL × 1.5 immediately

NOISE FILTER (before acting on any move):
  volume < 1.5× average = noise = HOLD
  volume > 1.5× average = signal = ACT
```

Entry Filters (ALL must pass)

```
confidence >= MIN_CONFIDENCE (default 0.68)
regime matches strategy
VIX between 10-30 (configurable, hard stop at 30)
time within 9:30-1:30 PM window
not already in trade (max 1 position default)
kill switch not active
capital gate check (drawdown status = NORMAL or HALF)
premium within configured range (₹80-200 Mode A, >₹300 Mode B)
bid_ask_spread <= MAX_SPREAD    ← liquidity gate (NEW)
open_interest >= MIN_OI         ← liquidity gate (NEW)
```

Charge-Aware Targeting

```
Nifty 1 lot, ₹150 premium entry:
  Fixed charges:    ~₹74
  Slippage est:     ~₹65 (₹1 × 65 units)
  Total round trip: ~₹139

  To net ₹1,000: need gross ₹1,139
  Per unit: ₹1,139 / 65 = ₹17.5 gain
  % of ₹150 premium: 11.7% gain needed
  Nifty move (delta 0.5): ~35 points

  Model phase1_target must exceed ₹139 minimum to be viable
```

SECTION 11 — ORDER EXECUTION

Market Order Flow

```
# Entry:
order = place_market_order(
```

```
    symbol     = groww_symbol,
    qty        = lots × lot_size,
    side       = "BUY",
    order_type = "MARKET"
)
# Fills instantly at best available price
fill_price = order.traded_price   # actual fill, not estimate

# Immediately after fill confirmation:
slm_order = place_slm_order(
    symbol        = groww_symbol,
    qty           = lots × lot_size,
    side          = "SELL",
    trigger_price = fill_price - sl_points,  # model decides sl_points
    order_type    = "SL-M"
)
trade.slm_order_id = slm_order.order_id

# Exchange now holds the SL — EC2 crash cannot hurt position
```

Trailing SL Amend Flow

```
# Every trail_tf minutes (model decided: 1, 5, or 15):
def update_trailing_sl(trade, new_candle):
    new_sl = new_candle.low - trade.trail_points

    if new_sl > trade.current_sl:
        groww.amend_order(
            order_id      = trade.slm_order_id,
            trigger_price = new_sl,
            order_type    = "SL-M"
        )
        trade.current_sl = new_sl
        log(f"SL moved: {trade.current_sl:.1f} → {new_sl:.1f}")

# Throttle: minimum 30 seconds between amends
# Max API calls: ~96/day (4 trades × 24 candles) — lightweight
```

Hard Exit

```
# 3:15 PM every day, no exceptions:
if current_time >= "15:15" and trade.is_open:
    place_market_order(side="SELL")   # close position
    cancel_slm_order(trade.slm_order_id)
```

```
    log("Hard exit: 3:15 PM")
```

SECTION 12 — POSITION SIZING

Kelly Criterion Auto-Scaling

```
# core/risk/position_sizer.py

def calculate_lots(capital, option_premium, confidence,
                   win_rate, avg_win, avg_loss, symbol):
    lot_size = LOT_SIZES[symbol]

    # Kelly fraction
    kelly = ((win_rate × avg_win) -
             ((1 - win_rate) × avg_loss)) / avg_win
    safe_kelly = kelly × 0.5      # half Kelly (conservative)

    # Confidence adjustment (0.70 = neutral)
    confidence_mult = confidence / 0.70
    adjusted = min(safe_kelly × confidence_mult, 0.20)

    # Capital to deploy
    deploy = capital × adjusted
    cost_per_lot = option_premium × lot_size
    lots = max(1, int(deploy / cost_per_lot))

    # Ceiling from DB (you control manually)
    return min(lots, get_ceiling(capital))

def get_ceiling(capital):
    if capital < 75_000:    return 1
    if capital < 1_50_000:  return 2
    if capital < 3_00_000:  return 4
    if capital < 5_00_000:  return 6
    if capital < 10_00_000: return 10
    return 15  # absolute max
```

Drawdown Protection

```
# core/risk/capital_tracker.py
def should_reduce_size(self):
    dd = self.drawdown_pct_from_peak
```

```
    if dd > DRAWDOWN_STOP_PCT:     # 25% default
        return "STOP"              # alert + halt
    elif dd > DRAWDOWN_HALF_PCT:   # 15% default
        return "HALF"              # halve lots
    return "NORMAL"
```

SECTION 13 — DB-BACKED CONFIG

agent_config Table

KEY	DEFAULT	CACHED	DESCRIPTION
INITIAL_CAPITAL	50000	yes	Starting capital
CAPITAL_INDEX_OPTION	50000	yes	Max per trade
LOT_SIZE_MULTIPLIER_NIFTY	1	yes	Manual ceiling override
LOT_SIZE_MULTIPLIER_SENSEX	1	yes	Manual ceiling override
DAILY_LOSS_LIMIT	5000	yes	Kill switch loss trigger
DAILY_TARGET	10000	yes	Profit halt trigger
KILL_SWITCH_ACTIVE	false	NO	← always direct DB read
ALLOW_OPTION_SELLING	false	NO	← always direct DB read
TRADING_MODE	paper	NO	← always direct DB read
EXPIRY_DAY_MODE	AUTO	yes	AUTO / A / B
MIN_CONFIDENCE	0.68	yes	Min to enter trade
MAX_TRADES_PER_DAY	4	yes	Swing trade limit
DRAWDOWN_HALF_SIZE_PCT	15.0	yes	Halve lots threshold
DRAWDOWN_STOP_PCT	25.0	yes	Stop trading threshold
PREMIUM_MIN	80	yes	Mode A min premium
PREMIUM_MAX	200	yes	Mode A max premium
KELLY_FRACTION	0.50	yes	0.5 = half Kelly
MAX_LOTS_HARD_CEILING	15	yes	Absolute max lots
SCALPING_ENABLED	false	yes	Auto-flips at gate
SCALPING_CAPITAL_GATE	300000	yes	₹3L to unlock
SHADOW_MODE	false	NO	← always direct DB read
ML_SIGNAL_SECRET	<secret>	yes	Auth for /signals/ml
VIX_EXTREME_CUTOFF	30	yes	No trades above this
VIX_HIGH_MIN_SL_POINTS	35	yes	Min SL when VIX > 25
VIX_LOW_MAX_SL_POINTS	20	yes	Max SL when VIX < 12
VIX_SPIKE_INTRADAY_PCT	5	yes	Trigger SL widen
VIX_SPIKE_SL_MULTIPLIER	1.5	yes	Widen factor on spike
MAX_BID_ASK_SPREAD_NIFTY	5.0	yes	Liquidity gate ₹
MAX_BID_ASK_SPREAD_SENSEX	8.0	yes	Liquidity gate ₹
MIN_OI_CONTRACTS	500	yes	Min open interest
SLIPPAGE_BUFFER_PCT	0.5	yes	Extra buffer in charge calc
MIN_SL_POINTS_HARD	5	yes	Model cannot go below

```
MAX_SL_POINTS_HARD            60        yes      Model cannot go above
MIN_TRAIL_POINTS_HARD         3         yes      Model cannot go below
MAX_TRAIL_POINTS_HARD         40        yes      Model cannot go above
```

Cache Rule

```
Cache TTL: 60 seconds for most settings
NEVER cache: KILL_SWITCH_ACTIVE, ALLOW_OPTION_SELLING,
             TRADING_MODE, SHADOW_MODE
These 4 keys always read directly from DB
```

Secrets in .env (never in DB)

```
DATABASE_URL
GROWW_API_KEY
GROWW_TOTP_SECRET
FYERS_CLIENT_ID
FYERS_SECRET_KEY
FYERS_REDIRECT_URI
FYERS_ID
FYERS_PIN
FYERS_TOTP_SECRET
TELEGRAM_BOT_TOKEN
TELEGRAM_CHAT_ID
ML_SIGNAL_SECRET
```

SECTION 14 — MANUAL TRADE HANDLING

Automatic Detection (Already Built)

```
monitering_service already has external trade detection.
Scans Groww positions every 60 seconds.
If position found NOT in our DB:
  → create Trade(source=EXTERNAL)
  → compute buy_price from avg_cost
  → compute initial SL = buy_price - (model default sl_points)
  → place SL-M on exchange immediately
  → register with TradeManager
  → same trailing SL amend logic applies
  → Telegram: "📱 EXTERNAL TRADE DETECTED — SL-M active"
```

Manual Entry via Dashboard

```
POST /trades/manual
Header: Authorization: Bearer <admin_token>
Body:
{
  "symbol":      "NIFTY25APR2522500CE",
  "lots":        1,
  "entry_price": 145.0,
  "sl_price":    110.0,
  "notes":       "Manual entry — saw hammer at support"
}

Response: trade_id + "Agent now managing this trade"
Agent: places SL-M at sl_price, trails per model rules,
       sends exit alerts, logs to DB + parquet
```

SECTION 15 — LIVE TRADE SIGNAL ENDPOINT

POST /signals/ml

```
Auth: X-Agent-Secret header (ML_SIGNAL_SECRET from .env)

Payload:
{
  "symbol":           "NIFTY",
  "action":           "BUY",        # BUY / SELL / EXIT / NO_TRADE
  "confidence":       0.74,
  "regime":           "uptrend",
  "strategy":         "hybrid_alpha",
  "signal":           1,
  "mode":             "MODE_A",
  "premium":          145.0,
  "strike":           22500,
  "expiry":           "2026-04-15",
  "dte":              3,
  "ce_iv":            14.2,
  "ce_delta":         0.52,
  "sl_bin":           "MEDIUM",
  "trail_bin":        "TIGHT",
  "phase1_target":    28.0,
  "trail_tf_minutes": 5,
  "vix_at_signal":    16.4,
```

```
  "vix_regime":       "NORMAL",
  "volatility_score": 0.48,
  "timestamp":        "2026-04-11T09:30:00+05:30"
}

EC2 handler order:
  1. Validate X-Agent-Secret
  2. Check KILL_SWITCH_ACTIVE (direct DB, NO cache)
  3. Check TRADING_MODE (direct DB, NO cache)
     shadow → log to shadow_trades, no order
     live → proceed
  4. Validate confidence >= MIN_CONFIDENCE
  5. Check VIX hard rules (VIX > 30 → reject)
  6. Check liquidity gate (bid-ask, OI)
  7. Calculate lots via position_sizer
  8. Get option symbol from option_chain service
  9. Convert symbol: fyers_to_groww()
  10. Place MARKET order → fill instantly
  11. Convert sl_bin + vix_score → actual sl_points
  12. Place SL-M order at (fill_price - sl_points)
  13. Register with TradeManager (store slm_order_id)
  14. Telegram: ENTRY alert with VIX regime
```

SECTION 16 — ANTI-OVERFITTING FRAMEWORK

Layer 1 — Data Integrity

```
# Strict no-lookahead: every label computable at bar close only
# WRONG: label = max_premium_next_20_bars > entry × 1.15
# RIGHT: label = premium_at_close_of_bar_N+4 > entry × 1.15

# Temporal split only — NEVER random shuffle:
train = data[:"2024-06-30"]          # oldest 80%
val   = data["2024-07-01":"2024-12-31"]  # middle 10%
test  = data["2025-01-01":]          # newest 10%
# Test set touched EXACTLY ONCE at final evaluation
```

Layer 2 — Walk-Forward Validation

```
# 5 folds, each trains on all past, validates on next quarter
FOLD 1: Train Jan23-Dec23 → Validate Jan24-Mar24
FOLD 2: Train Jan23-Mar24 → Validate Apr24-Jun24
```

```
FOLD 3: Train Jan23-Jun24 → Validate Jul24-Sep24
FOLD 4: Train Jan23-Sep24 → Validate Oct24-Dec24
FOLD 5: Train Jan23-Dec24 → Validate Jan25-Mar25

# Accept model ONLY if:
fold_win_rates must satisfy:
  mean > 0.55
  coefficient_of_variation < 0.08   # < 8% variance
  no single fold below 0.50
```

Layer 3 — Model Constraints

```
xgb_params = {
    "max_depth":            4,     # never > 6 for financial data
    "min_child_weight":     50,    # min 50 samples per leaf
    "reg_alpha":            0.1,   # L1 regularization
    "reg_lambda":           1.0,   # L2 regularization
    "gamma":                0.1,   # min gain to split
    "subsample":            0.8,   # 80% rows per tree
    "colsample_bytree":     0.7,   # 70% features per tree
    "learning_rate":        0.05,  # slow learner
    "n_estimators":         300,
    "early_stopping_rounds": 30,
}

lstm_params = {
    "dropout":            0.3,
    "recurrent_dropout":  0.2,
    "kernel_regularizer": l2(0.01),
    "final_dropout":      0.4,
    "patience":           15,      # early stopping
}
```

Layer 4 — Feature Control

```
# SHAP importance computed on VALIDATION set (not training)
# Unstable features dropped:
# Feature stable if SHAP rank variance < 20 positions across folds

# Minimum samples per feature bucket:
MIN_SAMPLES_PER_BUCKET = 100
```

Layer 5 — VIX-Aware Label Design

```
# SL bins (not exact values — prevents memorization):
SL_BINS = {
    "TIGHT":     (0,  10),
    "MEDIUM":    (10, 20),
    "WIDE":      (20, 35),
    "VERY_WIDE": (35, 60),
}

# VIX regime balance required:
assert vix_counts["LOW"]     >= 150
assert vix_counts["NORMAL"]  >= 300
assert vix_counts["HIGH"]    >= 100
assert vix_counts["EXTREME"] >= 30
```

Layer 6 — Live Monitoring

```
def detect_overfit_in_production(trades_df):
    # Win rate decay check
    recent_wr   = trades_df.tail(30).net_pnl.gt(0).mean()
    baseline_wr = trades_df.head(30).net_pnl.gt(0).mean()
    if recent_wr < baseline_wr - 0.10:
        alert("⚠️ Win rate degrading")

    # Confidence calibration check
    high_conf = trades_df[trades_df.confidence > 0.75]
    if high_conf.net_pnl.gt(0).mean() < 0.55:
        alert("⚠️ Model overconfident — retrain needed")

    # VIX regime drift check
    if current_vix_regime not in model.trained_vix_regimes:
        alert("⚠️ Unseen VIX regime — use wider SL")
```

Layer 7 — Retrain Guard

```
def should_deploy_new_model(old_model, new_model, val_data):
    old_wr = evaluate(old_model, val_data)["win_rate"]
    new_wr = evaluate(new_model, val_data)["win_rate"]

    if new_wr < old_wr + 0.02:      # < 2% better → keep old
        return False
    if abs(new_wr - old_wr) > 0.15: # > 15% change → suspicious
        alert("⚠️ Suspicious model jump — manual review")
        return False
    return True
```

SECTION 17 — MODEL IMPROVER

What It Is

```
NOT paper trading simulation.
YES: learning laboratory + data engine.

Runs locally during market hours.
Fetches real Fyers data → generates real ML signals.
Simulates exactly what would happen → logs everything.
Uses actual mid-price (bid+ask)/2 for entry simulation.
Deducts real charges + slippage estimate per trade.
Generates weekly report → retrain model → improve.
VIX regime tracked per trade for balanced retraining.
```

Trade Log (40+ fields to parquet)

```
FIELDS = [
    # Identity
    "trade_id", "model_version", "timestamp_entry", "timestamp_exit",
    # Instrument
    "symbol", "option_type", "strike", "expiry", "dte_at_entry",
    # Signal
    "signal", "action", "confidence", "regime", "strategy", "mode",
    # Model SL/Trail decisions
    "sl_bin", "trail_bin", "phase1_target", "trail_tf_minutes",
    "sl_points_used", "trail_points_used",
    # Pricing
    "entry_price", "exit_price", "premium_at_entry",
    "ce_iv_entry", "ce_delta_entry", "ce_theta_entry",
    # Position
    "lots", "lot_size", "capital_deployed", "capital_at_entry",
    # Outcome
    "gross_pnl", "charges_total", "slippage_estimate",
    "net_pnl", "pnl_pct",
    "was_model_right", "predicted_signal", "actual_outcome",
    "gave_back_pct",           # (max_profit - exit_profit) / max_profit
    # Phase tracking
    "phase_reached",           # HUNT / SECURED / TRAILING
    "hit_phase1", "hit_phase2", "hit_phase3",
    "max_profit_seen",
    "exit_reason",             # SL_HIT/TIME/REVERSAL/TRAIL/KILL
```

```
    # VIX context
    "vix_at_entry",
    "vix_regime_at_entry",     # LOW/NORMAL/HIGH/EXTREME
    "vix_change_during_trade",
    "vix_spike_occurred",      # bool: VIX moved 5%+ during trade
    "volatility_score",        # composite 0-1 at entry
    "sl_widened_by_vix",       # bool: hard rule triggered?
    # Time context
    "time_of_day", "day_of_week", "bars_to_expiry", "mode_used",
    # Charges breakdown
    "stt", "brokerage", "exchange_charges", "gst", "stamp_duty",
]
```

Graduation Criteria

```
GRADUATION = {
    "min_trades":         100,   # completed trades
    "min_win_rate":       0.55,  # 55%+ after charges
    "min_weeks":          6,     # minimum 6 weeks
    "max_drawdown_pct":   20.0,  # simulated drawdown
    "min_sharpe":         1.0,   # Sharpe ratio
    "must_be_profitable": True,  # positive net after charges
    "must_test_regimes":  True,  # trending + sideways seen
    "must_test_vix":      True,  # LOW + NORMAL + HIGH seen
    "max_gave_back_pct":  0.35,  # avg gave_back < 35%
}
# On graduation: Telegram "🎓 MODEL READY FOR LIVE"
```

SECTION 18 — DEPLOYMENT PIPELINE

```
STAGE 1 — Model Improver (6+ weeks, local)
  Run model_improver/ locally, collect 100+ trades
  All graduation criteria must pass
  VIX regime balance must be satisfied

STAGE 2 — Shadow Mode (1-2 weeks)
  Set SHADOW_MODE=true in DB via dashboard
  Signals sent to EC2 → logged but NOT executed
  Verify: signals arrive, EC2 healthy, VIX rules fire

STAGE 3 — Micro Live (2 weeks)
  Set TRADING_MODE=live in DB
```

```
  MAX_LOTS_HARD_CEILING=1, MAX_TRADES_PER_DAY=1
  Compare Model Improver P&L vs actual → should match

STAGE 4 — Full Live
  Remove restrictions, all agents active
  Kelly sizing at full, all strategies enabled
  ALLOW_OPTION_SELLING=true when ₹2L+ capital
```

SECTION 19 — ALL 23 GAPS

Critical — Fix Before Any Live Trading

```
GAP 11: Symbol converter
  Fix: fyers_to_groww() = symbol.split(":")[1] if ":" in symbol else symbol
  File: core/utils/symbol_converter.py

GAP 22: /signals/ml no authentication
  Fix: X-Agent-Secret header validated on EC2
  Both sides read ML_SIGNAL_SECRET from .env

GAP 23: Kill switch caching danger
  Fix: KILL_SWITCH_ACTIVE, TRADING_MODE, ALLOW_OPTION_SELLING, SHADOW_MODE
  → Always read directly from DB, never cache these 4

GAP 10: Timezone enforcement
  Fix: all datetime = pytz.timezone("Asia/Kolkata") everywhere

GAP 15: Charge + slippage accuracy in Model Improver
  Fix: charge_calculator.py with exact formula + slippage per instrument

GAP 25: Liquidity checker (NEW)
  Fix: check bid-ask spread + OI before every market order
  File: core/utils/liquidity_checker.py

GAP 26: Slippage estimator (NEW)
  Fix: per-instrument slippage in charge calculator + Model Improver
  File: core/utils/slippage_estimator.py
```

High — Fix Before Graduation

```
GAP 1:  Historical options data → option_premium_history.py
GAP 5:  Historical Greeks → calculate via mibian for all training bars
```

```
GAP 8:  Lot sizes updated (65/30/20) — use config/instruments.py everywhere
GAP 17: Token refresh in Model Improver (Fyers expires at midnight)
GAP 18: Shadow mode design (SHADOW_MODE DB flag + shadow_trades table)
GAP 20: Alembic migrations for all new tables
GAP 27: VIX features pipeline (NEW)
        vix_features.py + volatility_composite score in build_dataset.py
GAP 28: SL/trail bin labels in training data (NEW)
        label generation in build_dataset.py — distribution-based, not per-trade
```

Medium — Fix During Model Improver Phase

```
GAP 7:  Weekly retrain cron (scripts/weekly_retrain.py, Sunday 6 PM)
GAP 6:  Multi-agent conflict rules (priority order defined)
GAP 9:  Multi-leg orders for option selling (when capital gate met)
GAP 19: NSE market holiday calendar (market_calendar.py)
GAP 4:  Missing bar detection (gap fill in feature engineering)
GAP 16: Slippage — use actual traded_price from order response
GAP 29: Confidence calibration check in reporter.py (NEW)
        Platt scaling if calibration curve diverges
GAP 30: VIX regime balance check in weekly_retrain.py (NEW)
        Assert minimum trades per VIX regime before retraining
```

Low — Add When Capital Grows

```
GAP 2:  GIFT Nifty pre-market (gift_nifty.py)
GAP 3:  FII/DII daily data (fii_dii.py from NSE)
GAP 21: Model on Railway as backup
```

SECTION 20 — BUILD ORDER (41 Steps)

```
PHASE 1 — SETUP (Days 1-2)
  1.  mkdir nifty_agent + full folder structure
  2.  git init model_improver/ and live_trade/ separately
  3.  python3 -m venv .venv + pip install all dependencies
  4.  Copy fyers_client.py → core/data/ + add 4 methods
  5.  Copy option_chain.py → core/data/ + 4 fixes
  6.  Copy strategies.py → core/strategy/strike_selector.py + mode awareness
  7.  Write config/instruments.py (lot sizes, expiry days)
  8.  Write core/utils/symbol_converter.py (3 lines)
  9.  Write core/utils/charge_calculator.py (charges + slippage)
  10. Write core/utils/liquidity_checker.py (bid-ask + OI gate)
```

```
  11. Write core/utils/slippage_estimator.py (per-instrument)
  12. Write core/utils/market_calendar.py (NSE holidays)
  13. Create .env template

PHASE 2 — DATA DOWNLOAD (Days 3-5)
  14. Write scripts/download_data.py (all timeframes + options)
  15. Run download_data.py (takes 3-4 hours, check output)
  16. Write core/data/multi_tf_builder.py (resample + align)
  17. Write core/data/synthetic_premium.py (BSM Phase 1 fallback)
  18. Write core/data/option_premium_history.py (Phase 2 real data)

PHASE 3 — FEATURES (Days 6-8)
  19. Adapt features/engineering.py from nifty_ml_model
  20. Adapt features/regime.py from nifty_ml_model
  21. Write core/greeks/calculator.py (mibian BSM)
  22. Write core/features/candlestick.py (TA-Lib 10 patterns)
  23. Write core/features/pattern_context.py (context scorer)
  24. Write core/features/option_features.py (23 features)
  25. Write core/features/vix_features.py (8 VIX features + composite)
  26. Write scripts/build_dataset.py (merge all → parquet + SL/trail labels)
  27. Run build_dataset.py + verify shape + null check + VIX regime balance

PHASE 4 — MODEL (Days 9-12)
  28. Adapt core/model/train.py (direction + sl_bin + trail_bin outputs)
  29. Adapt core/model/research.py
  30. Train V1→V2→V3→V4→V5→V6 (add VIX SL learning last)
  31. Walk-forward validation (5 folds, consistency check)
  32. SHAP feature selection on VAL set (keep top 80)
  33. Write core/model/predict.py (live inference, outputs sl_bin + trail_bin)
  34. Write core/model/calibrator.py (Platt scaling)

PHASE 5 — MODEL IMPROVER (Days 13-20)
  35. Write model_improver/engine.py (main loop)
  36. Write model_improver/journal.py (40+ field parquet writer inc VIX fields)
  37. Write core/risk/position_sizer.py (Kelly + ceiling)
  38. Write core/risk/capital_tracker.py (drawdown protection)
  39. Write model_improver/reporter.py (weekly report + calibration check)
  40. Write core/model/promoter.py (graduation criteria inc VIX + gave_back)
  41. Write core/notifications/telegram.py (alerts)
  42. RUN Model Improver for 6+ weeks (iterate model weekly)

PHASE 6 — LIVE TRADE (Days 21-30, after graduation)
  43. Copy monitering_service code → live_trade/ (brokers/engine/risk)
  44. Update groww/adapter.py → place_market_order() + place_slm_order()
                                + amend_slm_order()
  45. Update engine/trade_monitor.py → SL-M amend per candle
  46. Update engine/stoploss_strategy.py → model-driven SL/trail
```

```
47. Write live_trade/api/ml_signal_router.py (inc VIX hard rules)
48. Write live_trade/services/ml_signal_service.py
49. Write live_trade/services/manual_trade_service.py
50. Write live_trade/config/agent_config.py (DB-backed + VIX config keys)
51. Write live_trade/api/config_router.py (UI control)
52. Extend models/trade.py (12 new fields + VIX fields)
53. Add Alembic migrations (alembic revision + upgrade head)
54. Write model_improver/shadow_mode.py
55. Deploy to EC2 (rsync + uvicorn)
56. Test /signals/ml endpoint (VIX hard rules, liquidity gate)
57. Run shadow mode 1-2 weeks
58. Run micro live 2 weeks
59. Full live 🎉
```

SECTION 21 — ALL COMMANDS (Run Yourself)

Initial Setup

```
mkdir -p nifty_agent && cd nifty_agent
mkdir -p core/{data,features,model/artifacts,strategy,risk,greeks,notifications,u
mkdir -p model_improver/data
mkdir -p live_trade/{api,services,brokers/groww,engine,models,repository,config}
mkdir -p scripts config data notebooks logs

cd model_improver && git init && echo "# Model Improver" > README.md && cd ..
cd live_trade && git init && echo "# Live Trade" > README.md && cd ..

python3 -m venv .venv
source .venv/bin/activate

# macOS:
brew install ta-lib
# Ubuntu:
# sudo apt-get install libta-lib-dev

pip install \
  fyers-apiv3 growwapi pyotp \
  pandas numpy scikit-learn xgboost \
  tensorflow keras \
  mibian TA-Lib \
  fastapi uvicorn sqlalchemy asyncpg \
  alembic pydantic pydantic-settings \
  pyarrow fastparquet \
```

```
  requests httpx aiohttp \
  python-dotenv pytz \
  joblib shap \
  python-telegram-bot \
  nsepy yfinance \
  pytest pytest-asyncio \
  black isort

python -c "import talib; print('TA-Lib ✅')"
python -c "import mibian; print('mibian ✅')"
python -c "import fyers_apiv3; print('Fyers ✅')"
python -c "import growwapi; print('Groww ✅')"
python -c "import shap; print('SHAP ✅')"
```

Data Download & Build

```
PYTHONPATH=. python scripts/download_data.py

python -c "
import pandas as pd, os
files = ['nifty_15m','nifty_60m','nifty_1d',
         'sensex_15m','option_premiums','india_vix_daily']
for f in files:
    path = f'data/{f}.parquet'
    if os.path.exists(path):
        df = pd.read_parquet(path)
        print(f'{f:25} {len(df):>8,} rows  {df.shape[1]:>4} cols')
    else:
        print(f'{f:25} NOT FOUND')
"

PYTHONPATH=. python scripts/build_dataset.py

python -c "
import pandas as pd
df = pd.read_parquet('data/training_data_final.parquet')
print('Shape:      ', df.shape)
print('Date range: ', df.index.min(), 'to', df.index.max())
print('Nulls:      ', df.isnull().sum().sum())
print('VIX regimes:', df['vix_regime'].value_counts().to_dict())
print('SL bins:    ', df['sl_bin'].value_counts().to_dict())
print('Memory:     ', df.memory_usage(deep=True).sum() / 1e6, 'MB')
"
```

Model Training

```
PYTHONPATH=. python scripts/train_model.py
PYTHONPATH=. python scripts/research_model.py

python -c "
import json
with open('core/model/artifacts/metrics.json') as f:
    m = json.load(f)
for k, v in m.items():
    print(f'{k}: {v}')
"
```

Model Improver

```
PYTHONPATH=. python model_improver/engine.py

# All-time stats
python -c "
import pandas as pd
df = pd.read_parquet('model_improver/data/trades.parquet')
print(f'Total trades:     {len(df)}')
print(f'Win rate:         {(df.net_pnl > 0).mean():.1%}')
print(f'Total PnL:        ₹{df.net_pnl.sum():,.0f}')
print(f'Avg gave_back:    {df.gave_back_pct.mean():.1%}')
print(f'VIX LOW trades:   {(df.vix_regime_at_entry==\"LOW\").sum()}')
print(f'VIX NORMAL:       {(df.vix_regime_at_entry==\"NORMAL\").sum()}')
print(f'VIX HIGH:         {(df.vix_regime_at_entry==\"HIGH\").sum()}')
print(f'VIX EXTREME:      {(df.vix_regime_at_entry==\"EXTREME\").sum()}')
"

# Check graduation
python -c "
from core.model.promoter import ModelPromoter
result = ModelPromoter().evaluate('model_improver/data/trades.parquet')
print('Ready for live:', result['ready_for_live'])
print('Win rate:      ', f\"{result['win_rate']:.1%}\")
print('Sharpe:        ', f\"{result['sharpe']:.2f}\")
print('Max drawdown:  ', f\"{result['max_drawdown']:.1f}%\")
print('Avg gave_back: ', f\"{result['avg_gave_back']:.1%}\")
print('VIX covered:   ', result['vix_regimes_seen'])
print('Missing:       ', result['missing'])
"
```

Live Trade Control (EC2)

```
EC2_URL="http://<EC2_IP>:8000"
SECRET="your_ml_signal_secret"

# Check kill switch
curl $EC2_URL/config/KILL_SWITCH_ACTIVE

# Emergency kill switch
curl -X PATCH $EC2_URL/config/KILL_SWITCH_ACTIVE \
  -H "Content-Type: application/json" \
  -d '{"value": "true"}'

# Check VIX config
curl $EC2_URL/config/VIX_EXTREME_CUTOFF

# Test signal (with VIX fields)
curl -X POST $EC2_URL/signals/ml \
  -H "Content-Type: application/json" \
  -H "X-Agent-Secret: $SECRET" \
  -d '{
    "symbol":           "NIFTY",
    "action":           "NO_TRADE",
    "confidence":       0.5,
    "signal":           0,
    "sl_bin":           "MEDIUM",
    "trail_bin":        "TIGHT",
    "vix_at_signal":    15.2,
    "vix_regime":       "NORMAL",
    "volatility_score": 0.42,
    "timestamp":        "'$(date -u +%Y-%m-%dT%H:%M:%S)'+05:30"
  }'
```

SECTION 22 — TELEGRAM ALERT MESSAGES

```
# Entry
"🟢 ENTRY\n"
"Symbol: NIFTY25APR2522500CE\n"
"Premium: ₹145 | Lots: 1 | Confidence: 74%\n"
"Mode: A | Regime: uptrend | DTE: 3\n"
"VIX: 16.4 (NORMAL) | SL: MEDIUM | Trail: TIGHT"

# Exit — profit
"✅ EXIT +₹1,842\n"
"Reason: PHASE3_TRAIL | Duration: 45min\n"
```

```
"Phase: TRAILING | Gave back: 18%\n"
"Win rate today: 2/3"

# Exit — loss
"❌ EXIT -₹1,350\n"
"Reason: SL_HIT | Duration: 12min\n"
"Phase: HUNT (never secured)\n"
"VIX at exit: 19.2 (spike during trade)"

# VIX hard rule triggered
"⚠️ VIX SPIKE DETECTED\n"
"VIX moved 6.2% intraday\n"
"Action: SL widened 1.5× on open trades\n"
"New SL: ₹108 (was ₹94)"

# VIX extreme — no trades
"🚫 TRADING PAUSED\n"
"India VIX: 31.4 (limit: 30)\n"
"No new entries until VIX normalises"

# Drawdown warning
"⚠️ DRAWDOWN WARNING\n"
"Current: 16% from peak\n"
"Action: LOTS HALVED (½ Kelly)\n"
"Capital: ₹42,300 | Peak: ₹50,360"

# Daily summary
"📊 DAILY SUMMARY — 11 Apr 2026\n"
"P&L: +₹2,340 | Trades: 3 (2W/1L)\n"
"Win Rate: 67% | Avg gave_back: 22%\n"
"VIX: 15.8 (NORMAL) | Capital: ₹52,340"

# Model graduated
"🎓 MODEL READY FOR LIVE\n"
"Version: v1.3 | Trades: 134\n"
"Win rate: 63% | Sharpe: 1.41\n"
"Avg gave_back: 21% | Max DD: 11%\n"
"VIX regimes: LOW ✅ NORMAL ✅ HIGH ✅\n"
"→ Switch TRADING_MODE=live in dashboard"

# Weekly retrain
"🔄 MODEL RETRAINED\n"
"v1.2 → v1.3\n"
"Win rate: 58% → 63% (+5%)\n"
"Gave_back: 31% → 22% (-9%) ✅\n"
"Deployed for next week"
```

SECTION 23 — REQUIREMENTS.TXT

```
# Broker APIs
fyers-apiv3>=3.1.11
growwapi>=1.0.0
pyotp>=2.9.0

# Data
pandas>=2.1.0
numpy>=1.26.0
pyarrow>=15.0.0
fastparquet>=2023.10.0
nsepy>=0.8
yfinance>=0.2.36

# ML / AI
scikit-learn>=1.4.0
xgboost>=2.0.0
tensorflow>=2.15.0
joblib>=1.3.0
shap>=0.44.0
TA-Lib>=0.4.28

# Options / Greeks
mibian>=0.1.3

# Web Framework
fastapi>=0.111.0
uvicorn>=0.29.0
httpx>=0.27.0
aiohttp>=3.9.0

# Database
sqlalchemy>=2.0.30
asyncpg>=0.29.0
alembic>=1.13.0

# Config / Validation
pydantic>=2.7.0
pydantic-settings>=2.3.0
python-dotenv>=1.0.0

# Timezone
pytz>=2024.1
```

```
# Alerts
python-telegram-bot>=20.0

# Utils
requests>=2.31.0

# Dev
pytest>=8.0.0
pytest-asyncio>=0.23.0
black>=24.0.0
isort>=5.13.0
```

SECTION 24 — .GITIGNORE

```
.venv/
.env
.env.*
data/
*.parquet
*.csv
*.joblib
.fyers_token
logs/
*.log
__pycache__/
*.pyc
*.pyo
.pytest_cache/
.vscode/
.DS_Store
core/model/artifacts/*.joblib
```

SECTION 25 — KEY DESIGN DECISIONS SUMMARY

```
WHAT MODEL DECIDES (learned from data):
  ✅ Entry direction (BUY/SELL/NO_TRADE)
  ✅ Confidence score
  ✅ SL width (bin: TIGHT/MEDIUM/WIDE/VERY_WIDE)
  ✅ Trail distance (bin: TIGHT/MEDIUM/WIDE)
  ✅ Phase 1 target (₹ amount)
  ✅ Trail timeframe (1/5/15 minutes)
```

```
WHAT YOU OWN (hardcoded safety rails):
  ✅ VIX > 30 = no trades (hard stop)
  ✅ VIX > 25 = min SL ₹35 (floor)
  ✅ VIX < 12 = max SL ₹20 (ceiling)
  ✅ VIX spike 5%+ = widen SL 1.5× immediately
  ✅ Min SL ₹5, Max SL ₹60 (absolute bounds)
  ✅ Min trail ₹3, Max trail ₹40 (absolute bounds)
  ✅ Hard exit 3:15 PM always
  ✅ Kill switch (manual override)
  ✅ Capital limits + drawdown halts

ORDER EXECUTION:
  ✅ Entry: MARKET order (instant fill)
  ✅ SL: SL-M on exchange (survives EC2 crash)
  ✅ Trail: amend SL-M per candle (1 API call)
  ✅ No limit orders anywhere
  ✅ No retry loops
  ✅ No GTT backup needed

ANTI-OVERFIT:
  ✅ 7 layers (data, walk-forward, constraints,
               features, labels, live monitoring, retrain guard)
  ✅ VIX regime balance enforced in training
  ✅ Binned labels (not exact values)
  ✅ gave_back_pct as key optimization metric
```

Document version: v3 | Market structure: post-Sept 2025 | Date: April 2026 Changes from v2: Market orders throughout, SL-M exchange-native execution, model-driven SL/trail (no hardcoded values), India VIX as primary SL driver, 7-layer anti-overfit framework, liquidity gate, slippage estimator, gave_back_pct metric, 23 gaps (down from 26), 41 build steps (down from 43) Feed this entire document to Claude AI in VS Code. Claude writes code. YOU run it.